DÉMO 11 · CHANGEMENTS ENTRE LES ÉTAPES

Refs

Chaque étape comparée à la précédente : ce qui est ajouté et ce qui est retiré.

`+ ligne ajoutée` `– ligne retirée` `@@ position dans le fichier`

1. **Étape 1 → 2** · etape-2-forwardref (3 fichiers)
2. **Étape 2 → 3** · etape-3-useimperativehandle (3 fichiers)

Étape 1 → 2 etape-1-useref → etape-2-forwardref

L'input devient un composant SearchInput qui reçoit la ref (React 19 : ref est une prop)

COMPARÉ `src/components/SearchRef.jsx → src/components/SearchForwardRef.jsx` +14 −8

```
@@ -1,20 +1,26 @@
- import { useRef } from "react";
+ import { forwardRef, useRef } from "react";

- function SearchRef() {
+ function SearchInput(props, ref) {
+   return (
+     <div>
+       <input ref={ref} type="text" placeholder="" />
+     </div>
+   );
+ }
+
+ const SearchInputForwarded = forwardRef(SearchInput);
+
+ function SearchForwardRef() {
    const inputRef = useRef(null);
-
    return (
      <div>
        <h1>{"Welcome to the Search Page"}</h1>
        <div className="container">
-         <div>
-           <input ref={inputRef} type="text" placeholder="" />
-         </div>
+         <SearchInputForwarded ref={inputRef} />
          <div>
            <button
              className="btn"
              onClick={() => {
-               console.log("Test", inputRef);
                inputRef.current?.focus();
              }}
            >
@@ -26,4 +32,4 @@
    );
  }

- export default SearchRef;
+ export default SearchForwardRef;
```

MODIFIÉ `src/App.jsx` +2 −1

```
@@ -1,10 +1,11 @@
 import SearchUseImperativeHandle from "./components/SearchUseImperativeHandle";
 import SearchRef from "./components/SearchRef";
+import SearchForwardRef from "./components/SearchForwardRef";

 import "./App.css";

 function App() {
-  return <SearchRef />;
+  return <SearchForwardRef />;
 }

 export default App;
```

UTILISÉ **src/components/SearchForwardRef.jsx** +35 −0

```
@@ -1,0 +1,35 @@
+import { forwardRef, useRef } from "react";
+
+function SearchInput(props, ref) {
+  return (
+    <div>
+      <input ref={ref} type="text" placeholder="" />
+    </div>
+  );
+}
+
+const SearchInputForwarded = forwardRef(SearchInput);
+
+function SearchForwardRef() {
+  const inputRef = useRef(null);
+  return (
+    <div>
+      <h1>{"Welcome to the Search Page"}</h1>
+      <div className="container">
+        <SearchInputForwarded ref={inputRef} />
+        <div>
+          <button
+            className="btn"
+            onClick={() => {
+              inputRef.current?.focus();
+            }}
+          >
+            Focus input
+          </button>
+        </div>
+      </div>
+    </div>
+  );
+}
+
+export default SearchForwardRef;
```

Étape 2 → 3 etape-2-forwardref → etape-3-useimperativehandle

useImperativeHandle : le parent n'a accès qu'à focus et clear

COMPARÉ **src/components/SearchForwardRef.jsx → src/components/SearchUseImperativeHandle.jsx** +40 −10

```
@@ -1,22 +1,44 @@
- import { forwardRef, useRef } from "react";
+ import { useImperativeHandle, useRef } from "react";

- function SearchInput(props, ref) {
+ // React 19 : ref est une prop normale, plus besoin de forwardRef
+ // correction : Ajout d'une ref interne pour l'input
+ const SearchInput = ({ ref }) => {
+   // IMPORTANT : Créer une ref INTERNE pour l'input
+   const inputRef = useRef(null);
+
+   // useImperativeHandle expose des méthodes custom au parent
+   useImperativeHandle(
+     ref,
+     () => ({
+       focus: () => {
+         inputRef.current?.focus(); // Utilise inputRef, pas ref
+       },
+       clear: () => {
+         if (inputRef.current) {
+           inputRef.current.value = "";
+         }
+       },
+       getValue: () => inputRef.current?.value || "", // Bonus : méthode ajoutée
+     }),
+     []
+   ); // [] : Pas de dépendances, on crée les méthodes une fois
+
    return (
      <div>
-       <input ref={ref} type="text" placeholder="" />
+       {/* Utilise inputRef interne, pas ref du parent */}
+       <input ref={inputRef} type="text" placeholder="" />
      </div>
    );
- }
+ };

- const SearchInputForwarded = forwardRef(SearchInput);
+ function SearchUseImperativeHandle() {
+   const inputRef = useRef(null);

- function SearchForwardRef() {
-   const inputRef = useRef(null);
    return (
      <div>
-       <h1>{"Welcome to the Search Page"}</h1>
+       <h1>Welcome to the Search Page</h1>
        <div className="container">
-         <SearchInputForwarded ref={inputRef} />
+         <SearchInput ref={inputRef} />
          <div>
            <button
              className="btn"
@@ -26,10 +48,18 @@
              >
              Focus input
```

```
28  50            </button>
    51          + <button
    52          +   className="btn"
    53          +   onClick={() => {
    54          +     inputRef.current?.clear();
    55          +   }}
    56          + >
    57          +   Clear input
    58          + </button>
29  59          </div>
30  60        </div>
31  61      </div>
32  62    );
33  63  }
34  64
35      - export default SearchForwardRef;
    65  + export default SearchUseImperativeHandle;
```

MODIFIÉ **src/App.jsx** +1 −2

```
    @@ -1,11 +1,10 @@
1   1   import SearchUseImperativeHandle from "./components/SearchUseImperativeHandle";
2   2   import SearchRef from "./components/SearchRef";
3       - import SearchForwardRef from "./components/SearchForwardRef";
4   3
5   4   import "./App.css";
6   5
7   6   function App() {
8       -   return <SearchForwardRef />;
    7   +   return <SearchUseImperativeHandle />;
9   8   }
10  9
11  10  export default App;
```

UTILISÉ **src/components/SearchUseImperativeHandle.jsx** +65 −0

```
    @@ -1,0 +1,65 @@
    1   + import { useImperativeHandle, useRef } from "react";
    2   +
    3   + // React 19 : ref est une prop normale, plus besoin de forwardRef
    4   + // correction : Ajout d'une ref interne pour l'input
    5   + const SearchInput = ({ ref }) => {
    6   +   // IMPORTANT : Créer une ref INTERNE pour l'input
    7   +   const inputRef = useRef(null);
    8   +
    9   +   // useImperativeHandle expose des méthodes custom au parent
    10  +   useImperativeHandle(
    11  +     ref,
    12  +     () => ({
    13  +       focus: () => {
    14  +         inputRef.current?.focus(); // Utilise inputRef, pas ref
    15  +       },
    16  +       clear: () => {
    17  +         if (inputRef.current) {
    18  +           inputRef.current.value = "";
    19  +         }
    20  +       },
    21  +       getValue: () => inputRef.current?.value || "", // Bonus : méthode ajoutée
    22  +     }),
    23  +     []
    24  +   ); // [] : Pas de dépendances, on crée les méthodes une fois
    25  +
    26  +   return (
    27  +     <div>
    28  +       {/* Utilise inputRef interne, pas ref du parent */}
```

```
+       <input ref={inputRef} type="text" placeholder="" />
+     </div>
+   );
+ };
+
+ function SearchUseImperativeHandle() {
+   const inputRef = useRef(null);
+
+   return (
+     <div>
+       <h1>Welcome to the Search Page</h1>
+       <div className="container">
+         <SearchInput ref={inputRef} />
+         <div>
+           <button
+             className="btn"
+             onClick={() => {
+               inputRef.current?.focus();
+             }}
+           >
+             Focus input
+           </button>
+           <button
+             className="btn"
+             onClick={() => {
+               inputRef.current?.clear();
+             }}
+           >
+             Clear input
+           </button>
+         </div>
+       </div>
+     </div>
+   );
+ }
+
+ export default SearchUseImperativeHandle;
```